

RELATÓRIO TÉCNICO DE ATIVIDADE PRÁTICA

ARQUITETURA E DESENVOLVIMENTO DE APIS

Centro Universitário Internacional UNINTER

Ano: 2026

Docentes: Prof. Pedro Henrique Flores e Prof. Osmar Dias Jr.

Estudante: João Paulo Andrade

RU: 5180040

Curso: Análise e Desenvolvimento de Sistemas

GitHub: <https://github.com/Joaopaulojpam/trabalho-api-uninter>

SUMÁRIO

1. [Introdução](#)
 2. [Tecnologias Utilizadas](#)
 3. [Justificativa das Escolhas](#)
 4. [Arquitetura da API](#)
 5. [Banco de Dados e ORM](#)
 6. [Endpoints e HTTP](#)
 7. [Tabela de Endpoints Obrigatória](#)
 8. [Validação de Dados e Tratamento de Erros](#)
 9. [Autenticação e CORS](#)
 10. [Testes e Evidências de Funcionamento](#)
 11. [Execução do Projeto](#)
 12. [Repositório Remoto](#)
 13. [Considerações Finais](#)
-

1. Introdução

1.1 Contextualização do Problema

No cenário contemporâneo de engenharia de software e transformação digital, a interoperabilidade entre aplicações heterogêneas é um requisito fundamental. Em ecossistemas de comércio eletrônico (*e-commerce*), sistemas integrados de gestão (*ERP*) e frentes de caixa (*PDV*), o catálogo de produtos e estoques representa o núcleo operacional dos negócios.

A ausência de uma camada de comunicação padronizada gera redundância de informações, vulnerabilidades de integridade concorrente e acoplamento excessivo. O problema abordado neste trabalho é a necessidade de disponibilizar uma plataforma centralizada, performática e padronizada que permita consultar, cadastrar, atualizar e remover itens de um catálogo de mercadorias, mantendo a consistência relacional e aplicando regras estritas de validação e controle de acesso.

1.2 Objetivo da Solução

O objetivo deste trabalho prático é planejar, arquitetar, implementar, documentar e demonstrar uma API Web autoral, orientada aos padrões da arquitetura RESTful, para o domínio de **Catálogo de Produtos e Categorias**.

A solução disponibiliza manipulação completa de recursos (*CRUD*), persistência em banco de dados relacional por meio de ORM, proteção de endpoints de alteração através de autenticação, política de CORS tecnicamente fundamentada, validação em tempo de requisição e geração automática de documentação interativa compatível com a especificação OpenAPI/Swagger.

1.3 Público e Cenário de Uso

A API foi projetada para ser consumida por: - **Aplicações Web (Frontends React/Vue/Angular)**: interfaces de painel administrativo (dashboard) e vitrines virtuais; - **Aplicativos Móveis (Android/iOS)**: aplicativos de vendas e inventário de estoque; - **Sistemas Terceiros e Microsserviços**: integração de faturamento e marketplaces parceiros.

2. Tecnologias Utilizadas

Para a concepção e implementação da solução, foram selecionadas ferramentas consolidadas na indústria:

- **Linguagem de Programação:** Python 3 (versão 3.14.6 / compatível com 3.10+).
- **Framework Web Backend:** FastAPI (v0.110+), construído sobre Starlette e Pydantic.
- **Servidor ASGI:** Uvicorn (v0.28+), servidor assíncrono para execução de alta disponibilidade.
- **Mapeamento Objeto-Relacional (ORM):** SQLAlchemy 2.0+, com uso da abordagem declarativa moderna.
- **Banco de Dados:** SQLite 3 (persistência baseada em arquivo físico `catalogo.db`).
- **Validação e Serialização (DTOs):** Pydantic v2, oferecendo validação estrita com tipagem estática.
- **Gerenciamento de Ambientes:** `python-dotenv` para injeção desacoplada de variáveis de ambiente.
- **Ferramentas de Testes:**
 - `pytest` (v9.1+): framework de testes automatizados unitários e de integração;
 - `httpx` / FastAPI `TestClient`: cliente HTTP assíncrono para testes de ponta a ponta (*E2E*).
- **Ferramentas de Documentação e Testes Manuais:** Swagger UI, ReDoc e Postman (coleção JSON inclusa).

3. Justificativa das Escolhas

A seleção da pilha tecnológica (*tech stack*) baseou-se em critérios técnicos objetivos de manutenibilidade, desempenho, aderência aos princípios REST e facilidade de reprodução pelo corpo docente:

1. **Python com FastAPI:**
2. **Performance e Tipagem:** O FastAPI figura entre os frameworks Python mais rápidos do mercado, aproveitando o padrão assíncrono ASGI e a validação do Pydantic em nível de compilação C/Rust.
3. **Documentação OpenAPI Nativa:** O FastAPI gera em tempo de execução a especificação OpenAPI 3.1 e as interfaces gráficas Swagger UI e ReDoc, eliminando discrepâncias entre o código e a documentação técnica.
4. **Injeção de Dependências:** O sistema de *Dependency Injection* nativo ([Depends](#)) permite desacoplar sessões de banco de dados e validações de segurança de forma limpa e modular.
5. **SQLAlchemy 2.0 como ORM:**
6. Permite abstrair a sintaxe SQL crua, mapeando tabelas relacionais em classes Python com tipagem.
7. Gerencia eficientemente o ciclo de vida das transações ([commit](#) , [rollback](#)), chaves estrangeiras, restrições de integridade e relacionamentos 1:N entre categorias e produtos.
8. **SQLite para Persistência:**
9. Garante **persistência real dos dados entre reinicializações da aplicação**, atendendo expressamente aos requisitos do edital.
10. Não requer a instalação e configuração de servidores de banco de dados pesados e externos (como PostgreSQL ou MySQL), viabilizando que o professor execute e avalie o projeto imediatamente em qualquer máquina com um único comando.
11. **Pytest com FastAPI TestClient:**
12. Permite testes automatizados rigorosos, cobrindo rotas válidas, situações de erro, validações e autenticação, com isolamento transacional completo através de banco em memória durante as baterias de testes.

4. Arquitetura da API

4.1 Organização Estrutural do Código

A aplicação foi estruturada seguindo o padrão modular por responsabilidades (*Separation of Concerns*), garantindo facilidade de extensão e desacoplamento:

TRABALHO DE API/

```
├─ app/
│   ├── __init__.py      # Inicialização do pacote principal
│   ├── config.py        # Configurações centralizadas e leitura do .env
│   ├── database.py      # Conexão de banco, SessionLocal e Base do ORM
│   ├── models.py        # Modelos relacionais ORM (Categoria e Produto)
│   ├── schemas.py       # Schemas Pydantic (validação de entrada e DTOs de saída)
│   ├── security.py      # Autenticação via API Key / Token Bearer
│   └─ routers/
│       ├── __init__.py
│       ├── produtos.py  # Endpoints CRUD do recurso principal (Produto)
│       └─ categorias.py # Endpoints do recurso Categoria (relacionamento)
├─ tests/
│   ├── __init__.py
│   └─ test_api.py      # Suíte completa com 27 testes automatizados (pytest)
├─ seed.py              # Script de recriação e povoamento inicial do banco
├─ run.py                # Script de inicialização facilitada da API
├─ test_requests.py     # Script interativo de demonstração de requisições HTTP
├─ postman_collection.json # Coleção Postman pronta para importação
├─ requirements.txt      # Lista de dependências do projeto
├─ .env.example         # Exemplo seguro de variáveis de ambiente
├─ .env                 # Variáveis ativas para o ambiente de desenvolvimento
├─ .gitignore           # Exclusões de arquivos temporários do versionamento Git
├─ README.md            # Instruções de instalação e execução conforme edital
└─ relatorio_tecnico.md # Relatório acadêmico completo com 12 seções
```

4.2 Fluxo Geral da Aplicação

1. O cliente HTTP (navegador, Postman, aplicação externa) emite uma requisição para a rota desejada;
2. O middleware de **CORS** intercepta a chamada e valida a origem, métodos e cabeçalhos permitidos;
3. O roteador ([APIRouter](#)) direciona a requisição para a função manipuladora;
4. Se o endpoint for protegido, a dependência de segurança ([verificar_autenticacao](#)) valida a presença e integridade do token no cabeçalho [X-API-Key](#) ou [Authorization: Bearer](#) ;
5. O corpo da requisição é validado estritamente pelo esquema **Pydantic** correspondente ([ProdutoCreate](#) , [ProdutoUpdate](#) , [ProdutoPatch](#));
6. A sessão do banco de dados é injetada via [get_db\(\)](#) ;
7. O modelo **SQLAlchemy** executa as operações de consulta ou persistência com integridade transacional;
8. A resposta é serializada em formato JSON padronizado com o código de status HTTP correspondente (200, 201, 204, etc.).

5. Banco de Dados e ORM

5.1 Entidades e Mapeamento Objeto-Relacional

O modelo de dados implementa duas entidades relacionais: **Categoria** e **Produto**, configuradas no arquivo `app/models.py` utilizando a classe declarativa `Base` do SQLAlchemy:

categorias		produtos	
PK id: INTEGER	1	PK id: INTEGER	N
nome: VARCHAR(100) [UNIQUE]	<-----	FK categoria_id: INTEGER	
descricao: VARCHAR(255)		nome: VARCHAR(150)	
criada_em: DATETIME		descricao: VARCHAR(500)	
		preco: FLOAT	
		estoque: INTEGER	
		codigo_sku: VARCHAR(50) [UQ]	
		ativo: BOOLEAN	
		criado_em: DATETIME	
		atualizado_em: DATETIME	

5.2 Funcionamento da Persistência

- **Relacionamento 1:N:** Uma categoria possui muitos produtos (`relationship("Produto", back_populates="categoria")`). Cada produto referencia sua respectiva categoria através da chave estrangeira `categoria_id = Column(Integer, ForeignKey("categorias.id", ondelete="RESTRICT"))`.
- **Integridade de Negócio:**
 - O campo `codigo_sku` (Stock Keeping Unit) possui restrição de unicidade (`unique=True, index=True`), impedindo duplicidade de códigos no estoque;
 - O campo `nome` em categorias também é único e indexado;
 - Campos de data/hora (`criado_em` e `atualizado_em`) utilizam *timestamps* UTC automáticos;
- **Persistência Real:** Os dados são gravados no arquivo físico `catalogo.db`. Qualquer operação efetuada permanece gravada mesmo que o processo do servidor seja reiniciado.

6. Endpoints e HTTP

6.1 Padrão de Projeto REST e Métodos Adotados

A API segue integralmente os princípios da arquitetura REST: - Identificação de recursos através de URIs com substantivos no plural (`/api/v1/produtos`, `/api/v1/categorias`); - Utilização explícita dos métodos HTTP para expressar ações sobre os recursos; - Comunicação sem estado (*stateless*); - Respostas orientadas ao formato padrão `application/json`.

6.2 Justificativa da Escolha entre PUT e PATCH

O edital estabelece a obrigatoriedade de implementar e justificar pelo menos um entre `PUT` e `PATCH`. Nesta solução, **foram implementados ambos**, proporcionando suporte abrangente às melhores práticas HTTP:

- **Método `PUT` (Atualização Integral / Idempotente):**
- **Justificativa:** O método `PUT` é empregado para a **substituição completa** do recurso. Quando o cliente deseja atualizar um produto, ele envia a representação integral de todos os atributos obrigatórios. Caso executado repetidas vezes com os mesmos dados, o estado final do recurso permanece inalterado (idempotência estrita).
- **Método `PATCH` (Atualização Parcial):**
- **Justificativa:** O método `PATCH` é empregado para **modificações cirúrgicas ou pontuais**. Em cenários práticos de e-commerce, operações rotineiras como alterar exclusivamente o preço de um produto ou atualizar a quantidade em estoque não devem obrigar o cliente a trafegar novamente todos os campos cadastrais (como descrições longas ou imagens). O `PATCH` processa apenas os atributos recebidos (`exclude_unset=True`), mantendo os demais intactos.

7. Tabela de Endpoints Obrigatória

Conforme modelo exigido nas diretrizes da Atividade Prática, a tabela abaixo lista todos os endpoints reais implementados no sistema:

Método	Endpoint	Finalidade	Entrada principal	Respostas esperadas
GET	/api/v1/produtos	Listar coleção de produtos com paginação e busca	Parâmetros de consulta opcionais (skip , limit , categoria_id , busca , preco_min , preco_max)	200 OK
GET	/api/v1/produtos/{id}	Consultar produto específico individual	Identificador numérico na URL (id)	200 OK / 404 Not Found
POST	/api/v1/produtos	Criar novo produto no catálogo (Protegido)	Cabeçalho X-API-Key + JSON com dados do produto	201 Created / 400 Bad Request / 401 Unauthorized / 404 Not Found / 409 Conflict / 422 Unprocessable
PUT	/api/v1/produtos/{id}	Atualização integral de um produto (Protegido)	Cabeçalho X-API-Key + id + JSON completo com atributos	200 OK / 400 Bad Request / 401 Unauthorized / 404 Not Found / 409 Conflict / 422 Unprocessable
PATCH	/api/v1/produtos/{id}	Atualização parcial de campos do produto (Protegido)	Cabeçalho X-API-Key + id + JSON com campos a alterar	200 OK / 400 Bad Request / 401 Unauthorized / 404 Not Found / 409 Conflict / 422 Unprocessable
DELETE	/api/v1/produtos/{id}	Excluir produto do catálogo (Protegido)	Cabeçalho X-API-Key + Identificador numérico na URL (id)	204 No Content / 401 Unauthorized / 404 Not Found
GET	/api/v1/categorias	Listar categorias cadastradas	Parâmetros de paginação opcionais (skip , limit)	200 OK
GET	/api/v1/categorias/{id}	Consultar categoria por ID	Identificador numérico na URL (id)	200 OK / 404 Not Found

Método	Endpoint	Finalidade	Entrada principal	Respostas esperadas
POST	<code>/api/v1/categorias</code>	Cadastrar nova categoria (<i>Protegido</i>)	Cabeçalho <code>X-API-Key</code> + JSON com nome e descrição	<code>201 Created</code> / <code>401 Unauthorized</code> / <code>409 Conflict</code> / <code>422 Unprocessable</code>
DELETE	<code>/api/v1/categorias/{id}</code>	Excluir categoria por ID (<i>Protegido</i>)	Cabeçalho <code>X-API-Key</code> + Identificador numérico na URL (<code>id</code>)	<code>204 No Content</code> / <code>401 Unauthorized</code> / <code>404 Not Found</code>

8. Validação de Dados e Tratamento de Erros

8.1 Estratégias de Validação com Pydantic

A API implementa validação em duas camadas: no nível da borda HTTP (esquemas de DTO com Pydantic) e no nível do banco de dados (restrições do ORM): - **Preço estritamente positivo:** `preco: float = Field(..., gt=0)` impede cadastro ou atualização com valores negativos ou zero; - **Estoque não-negativo:** `estoque: int = Field(0, ge=0)` assegura coerência física no estoque; - **Tamanho de campos de texto:** Comprimento mínimo e máximo para nomes e SKU; - **Higienização de Strings:** Aplicação de `.strip()` em nomes e `.upper()` no código SKU.

8.2 Respostas de Erro e Códigos de Status HTTP

Quando uma operação não pode ser concluída com sucesso, a API rejeita a requisição imediatamente, fornecendo mensagens explicativas:

- **400 Bad Request / 422 Unprocessable Entity:** Disparado automaticamente quando os tipos ou formatos enviados no JSON são incompatíveis com as regras de validação.
- **401 Unauthorized:** Disparado quando operações protegidas (POST, PUT, PATCH, DELETE) são acionadas sem a chave de autenticação ou com credencial incorreta. Inclui cabeçalho `WWW-Authenticate: ApiKey`.
- **404 Not Found:** Disparado quando o ID do recurso solicitado não existe no banco de dados (ex.: produto ou categoria inexistente).
- **409 Conflict:** Disparado quando há violação de unicidade, como tentativa de cadastrar um SKU repetido ou duplicar o nome de uma categoria.
- **204 No Content:** Resposta de sucesso para operações `DELETE`, sem envio de corpo redundante, conforme preceitua a especificação RFC 9110.

9. Autenticação e CORS

9.1 Mecanismo de Autenticação e Controle de Acesso

Para garantir a segurança das operações de escrita e manipulação de recursos, foi implementado um mecanismo de **Autenticação por Chave de API / Token Bearer**: - **Rotas de Leitura (GET)**: São públicas, permitindo consulta irrestrita ao catálogo de mercadorias. - **Rotas de Modificação (POST, PUT, PATCH, DELETE)**: São privadas e protegidas. Exigem o envio de um cabeçalho HTTP de autorização. - **Cabeçalhos Suportados**: 1. `X-API-Key: uninter_segredo_api_2026` 2. `Authorization: Bearer uninter_segredo_api_2026` - **Fluxo de Autenticação**: text [Cliente] ---> Envia requisição (ex: POST /api/v1/produtos) Com cabeçalho "X-API-Key: uninter_segredo_api_2026" | v [verificar_autenticacao] (Dependência FastAPI) | +---> A chave confere com a variável de ambiente API_KEY? | +--- SIM ---> Prossegue para execução da rota (201 Created) | +--- NÃO ---> Aborta a execução imediatamente Retorna HTTP 401 Unauthorized Header: "WWW-Authenticate: ApiKey" Body: {"detail": "Acesso não autorizado..."}

9.2 Política de CORS (Cross-Origin Resource Sharing)

O mecanismo de CORS é uma salvaguarda de segurança implementada nos navegadores para restringir o acesso a recursos hospedados em origens (domínio, protocolo ou porta) diferentes da origem de onde o script requisitante foi carregado.

Na aplicação, o `CORSMiddleware` foi configurado no arquivo `app/main.py`: - **Origens Permitidas (allow_origins)**: Configurável dinamicamente via variável de ambiente `CORS_ORIGINS`. Em desenvolvimento, o valor padrão é `*` para permitir que ferramentas locais de teste e clientes frontend em portas dinâmicas (ex: `localhost:3000`, `localhost:5173`) acessem a API sem bloqueios de *Same-Origin Policy*. Para produção, a variável pode ser facilmente restrita ao domínio do frontend oficial; - **Métodos Permitidos (allow_methods)**: `["GET", "POST", "PUT", "PATCH", "DELETE", "OPTIONS"]`; - **Cabeçalhos Permitidos (allow_headers)**: Permite cabeçalhos customizados, incluindo `X-API-Key`, `Authorization` e `Content-Type`.

10. Testes e Evidências de Funcionamento

10.1 Suíte de Testes Automatizados (Pytest)

A integridade da API foi validada através de uma suíte automatizada contendo **27 casos de teste**, implementada em `tests/test_api.py`. Todos os 27 testes foram executados com **100% de aprovação**:

```

===== test session starts =====
platform win32 -- Python 3.14.6, pytest-9.1.1, pluggy-1.6.0
rootdir: C:\Users\JP\OneDrive\Desktop\TRABALHO DE API
collected 27 items

tests/test_api.py::test_rota_raiz PASSED [ 3%]
tests/test_api.py::test_rota_health PASSED [ 7%]
tests/test_api.py::test_listar_categorias PASSED [ 11%]
tests/test_api.py::test_obter_categoria_por_id PASSED [ 14%]
tests/test_api.py::test_obter_categoria_inexistente PASSED [ 18%]
tests/test_api.py::test_criar_categoria_sem_auth PASSED [ 22%]
tests/test_api.py::test_criar_categoria_com_auth PASSED [ 25%]
tests/test_api.py::test_criar_categoria_nome_duplicado PASSED [ 29%]
tests/test_api.py::test_listar_produtos PASSED [ 33%]
tests/test_api.py::test_filtrar_produtos_por_categoria PASSED [ 37%]
tests/test_api.py::test_filtrar_produtos_por_faixa_preco PASSED [ 40%]
tests/test_api.py::test_buscar_produtos_por_termo PASSED [ 44%]
tests/test_api.py::test_obter_produto_por_id PASSED [ 48%]
tests/test_api.py::test_obter_produto_inexistente_retorna_404 PASSED [ 51%]
tests/test_api.py::test_criar_produto_sem_autenticacao_retorna_401 PASSED [ 55%]
tests/test_api.py::test_criar_produto_com_autenticacao_retorna_201 PASSED [ 59%]
tests/test_api.py::test_criar_produto_com_bearer_token_retorna_201 PASSED [ 62%]
tests/test_api.py::test_criar_produto_com_preco_invalido_retorna_422 PASSED [ 66%]
tests/test_api.py::test_criar_produto_categoria_inexistente_retorna_404 PASSED [ 70%]
tests/test_api.py::test_criar_produto_sku_duplicado_retorna_409 PASSED [ 74%]
tests/test_api.py::test_atualizar_produto_put_sucesso PASSED [ 77%]
tests/test_api.py::test_atualizar_produto_put_inexistente_retorna_404 PASSED [ 81%]
tests/test_api.py::test_atualizar_produto_patch_parcial_sucesso PASSED [ 85%]
tests/test_api.py::test_atualizar_produto_patch_sem_autenticacao_retorna_401 PASSED [ 88%]
tests/test_api.py::test_excluir_produto_sem_autenticacao_retorna_401 PASSED [ 92%]
tests/test_api.py::test_excluir_produto_sucesso_retorna_204 PASSED [ 96%]
tests/test_api.py::test_excluir_produto_inexistente_retorna_404 PASSED [100%]

===== 27 passed in 1.74s =====

```

10.2 Evidências Reais das Operações HTTP

Abaixo constam os registros literais de requisições e respostas emitidas durante o ciclo de demonstração ([test_requests.py](#)):

Evidência 1: Listagem de Produtos com Filtros (GET 200 OK)

- **Requisição:** `GET /api/v1/produtos?categoria_id=1&preco_min=100&preco_max=300`
- **Código de Resposta:** `200 OK`
- **Corpo da Resposta:**

```
[
  {
    "nome": "Mouse Gamer Ergonômico RGB",
    "descricao": "Mouse óptico com sensor de 12.000 DPI, 7 botões programáveis e cabo trançado.",
    "preco": 149.9,
    "estoque": 35,
    "codigo_sku": "INF-MOU-001",
    "categoria_id": 1,
    "ativo": true,
    "id": 1,
    "criado_em": "2026-09-16T09:45:27.758572",
    "atualizado_em": "2026-09-16T09:45:27.758574",
    "categoria": {
      "nome": "Informática",
      "descricao": "Computadores, periféricos e suprimentos de TI",
      "id": 1,
      "criada_em": "2026-09-16T09:45:27.755308"
    }
  },
  {
    "nome": "Teclado Mecânico ABNT2 Switch Blue",
    "descricao": "Teclado mecânico compacto 75%, iluminação Rainbow e teclas em double-shot injetado",
    "preco": 289.0,
    "estoque": 20,
    "codigo_sku": "INF-TEC-002",
    "categoria_id": 1,
    "ativo": true,
    "id": 2,
    "criado_em": "2026-09-16T09:45:27.758575",
    "atualizado_em": "2026-09-16T09:45:27.758575",
    "categoria": {
      "nome": "Informática",
      "descricao": "Computadores, periféricos e suprimentos de TI",
      "id": 1,
      "criada_em": "2026-09-16T09:45:27.755308"
    }
  }
]
```

Evidência 2: Tentativa de Criação Não Autorizada (POST 401 Unauthorized)

- **Requisição:** `POST /api/v1/produtos` (sem envio de cabeçalho de autenticação)
- **Código de Resposta:** `401 Unauthorized`
- **Corpo da Resposta:**

```
{
  "detail": "Acesso não autorizado: Chave de API ou Token Bearer inválido ou não fornecido."
}
```

Evidência 3: Criação de Recurso com Autenticação (POST 201 Created)

- **Requisição:** `POST /api/v1/produtos`
- **Cabeçalho:** `X-API-Key: uninter_segredo_api_2026`
- **Corpo Enviado:**

```
{
  "nome": "Webcam Pro 4K",
  "descricao": "Webcam Ultra HD com foco automático",
  "preco": 399.90,
  "estoque": 20,
  "codigo_sku": "INF-CAM-005",
  "categoria_id": 1,
  "ativo": true
}
```

- **Código de Resposta:** `201 Created`
- **Corpo Retornado:**

```
{
  "nome": "Webcam Pro 4K",
  "descricao": "Webcam Ultra HD com foco automático",
  "preco": 399.9,
  "estoque": 20,
  "codigo_sku": "INF-CAM-005",
  "categoria_id": 1,
  "ativo": true,
  "id": 8,
  "criado_em": "2026-09-16T09:46:32.616665",
  "atualizado_em": "2026-09-16T09:46:32.616669",
  "categoria": {
    "nome": "Informática",
    "descricao": "Computadores, periféricos e suprimentos de TI",
    "id": 1,
    "criada_em": "2026-09-16T09:45:27.755308"
  }
}
```

Evidência 4: Validação de Entrada Inválida (POST 422 Unprocessable Entity)

- **Requisição:** `POST /api/v1/produtos` com preço negativo e SKU vazio
- **Código de Resposta:** `422 Unprocessable Entity`
- **Corpo Retornado:**

```
{
  "detail": [
    {
      "type": "string_too_short",
      "loc": ["body", "nome"],
      "msg": "String should have at least 2 characters",
      "input": "X"
    },
    {
      "type": "greater_than",
      "loc": ["body", "preco"],
      "msg": "Input should be greater than 0",
      "input": -10.0
    }
  ]
}
```

Evidência 5: Atualização Parcial de Dados (PATCH 200 OK)

- **Requisição:** `PATCH /api/v1/produtos/8`
- **Cabeçalho:** `X-API-Key: uninter_segredo_api_2026`
- **Corpo Enviado:** `{"preco": 420.00, "estoque": 30}`
- **Código de Resposta:** `200 OK`
- **Corpo Retornado:** Reflete as alterações pontuais sem perder os dados originais (nome, descrição e SKU preservados).

Evidência 6: Exclusão de Recurso (DELETE 204 No Content)

- **Requisição:** `DELETE /api/v1/produtos/8`
- **Cabeçalho:** `X-API-Key: uninter_segredo_api_2026`
- **Código de Resposta:** `204 No Content` (sem corpo, conforme RFC).
- **Verificação subsequente:** `GET /api/v1/produtos/8 -> 404 Not Found` confirmando a remoção física no banco.

11. Execução do Projeto

Para reproduzir a execução da API em qualquer ambiente local, siga o procedimento:

1. **Instalação dos Módulos:** `bash pip install -r requirements.txt`
2. **Inicialização do Banco de Dados (Seed):** `bash python seed.py`
3. **Inicialização do Servidor:** `bash python run.py`
4. **Acesso à Documentação Interativa:** Acessar `http://127.0.0.1:8000/docs` no navegador web para testes visuais com Swagger UI.
5. **Execução da Suíte de Testes:** `bash python -m pytest tests/test_api.py -v`

12. Repositório Remoto

O código-fonte integral, documentação, testes e suíte de automação estão disponibilizados no seguinte repositório Git remoto acessível para avaliação:

- **URL do Repositório:** <https://github.com/joapaulojpam/trabalho-api-uninter>
-

13. Considerações Finais

13.1 Dificuldades Encontradas e Decisões Tomadas

Durante o planejamento e o desenvolvimento da aplicação, alguns desafios técnicos foram superados: 1. **Padronização da Autenticação sem Comprometer a Avaliação:** Optou-se por aceitar tanto o cabeçalho moderno `X-API-Key` quanto o padrão universal `Authorization: Bearer <chave>`. Essa flexibilidade permite que o avaliador teste as rotas tanto via cURL quanto diretamente pelo botão "Authorize" do Swagger UI sem qualquer dificuldade; 2. **Isolamento de Testes Automatizados:** Para garantir que os testes automatizados não alterassem a base de dados principal (`catalogo.db`), foi configurado um banco de dados SQLite em memória (`sqlite:///memory:`) exclusivo para a suíte do `pytest` , com injeção de dependência dinâmica no FastAPI; 3. **Implementação Simultânea de PUT e PATCH:** Embora o regulamento permitisse escolher apenas um, optou-se por implementar ambos os métodos com tratamento diferenciado, demonstrando a compreensão profunda das diferenças de idempotência e granularidade de payloads na arquitetura REST.

13.2 Limitações e Possíveis Melhorias

Para iterações futuras e evolução do produto em um cenário de produção em larga escala, destacam-se as seguintes melhorias: - Implementação de autenticação baseada em JWT (*JSON Web Tokens*) com expiração temporária e assinatura criptográfica assimétrica (RSA/ECDSA); - Criação de migrações automáticas de esquema de banco com Alembic; - Adição de limitação de taxa de requisições (*Rate Limiting*) para proteção contra ataques de negação de serviço (DoS); - Containerização completa da aplicação através de Docker e Docker Compose.

Declaração de Autoria: O presente trabalho foi concebido e implementado de forma autoral pelo estudante para fins avaliativos na disciplina de Arquitetura e Desenvolvimento de APIs da UNINTER.